

Self-Improving LLM Agents for Long-Horizon Tasks

Akash Gogate, Ajeenkya Mahadik, Pratik Sahu, Bingye Wu, Fangkai Yu

Department of Computer Science

University of Wisconsin–Madison

Email: {agogate, avmahadik, psahu, bwu226, fyu59}@wisc.edu

Abstract—Large Language Models (LLMs) have demonstrated remarkable capabilities in reasoning, planning, and tool usage. However, LLM-based agents struggle with long-horizon tasks that require multiple sequential actions. As task horizons increase, small reasoning errors accumulate, leading to cascading failures. In this paper, we propose a self-improving LLM agent framework that enables agents to analyze execution failures and generate corrective strategies without retraining the underlying model. Our system introduces a strategy memory mechanism that stores failure patterns and corresponding corrective strategies generated by a secondary LLM. During future executions, the agent retrieves relevant strategies and integrates them into its reasoning process. We evaluate the proposed system using long-horizon agent benchmarks including AgentBench and WebArena. Our approach improves task success rates and reduces repeated failure patterns compared to baseline agents such as ReAct and Plan-and-Act. The results demonstrate that agents can progressively improve through experience while operating purely at inference time.

I. INTRODUCTION

Large Language Models (LLMs) such as GPT-4, LLaMA, and Mistral have demonstrated impressive capabilities across a wide range of natural language processing tasks. Recently, researchers have extended these models into autonomous agents capable of interacting with external tools and environments.

LLM agents typically operate using a reasoning-action loop in which the model generates reasoning steps followed by actions that interact with an environment. While these agents perform well on short tasks, their performance degrades significantly as task complexity and length increase.

Long-horizon tasks require maintaining context across multiple reasoning steps, executing sequential decisions, and adapting to environment feedback. Errors made early in the process often propagate through later steps, resulting in cascading failures.

A key limitation of current LLM agents is the absence of mechanisms for learning from past failures. Each task execution is treated independently, meaning agents frequently repeat the same mistakes across different tasks.

In this paper, we introduce a *Self-Improving LLM Agent* framework that enables agents to analyze their own failures and generate corrective strategies without retraining the underlying model. Our contributions include:

- A failure analysis framework for identifying root causes from agent execution traces

- A strategy generation module that produces corrective policies
- A persistent strategy memory enabling cross-task learning
- Experimental evaluation on long-horizon benchmarks

The proposed system allows LLM agents to gradually improve performance through experience while operating entirely at inference time.

II. RELATED WORK

Recent research has explored extending LLMs into autonomous agents.

A. LLM Agents

ReAct [1] introduced a reasoning and acting framework where LLMs generate both reasoning traces and actions. This approach improves transparency and reasoning quality but does not incorporate mechanisms for learning from past failures.

Plan-and-Act frameworks [2] attempt to improve reliability by separating planning and execution phases. However, these approaches still treat each task independently.

B. Self-Improving Agents

Reflexion [3] proposed an approach where agents use self-reflection to improve future decisions. Voyager [4] demonstrated lifelong learning in Minecraft environments by building a skill library.

Our work builds on these ideas by focusing specifically on long-horizon reasoning tasks and introducing a persistent strategy memory for failure correction.

III. PROBLEM DEFINITION

We consider a task environment defined as

$$T = (S, A, O)$$

where:

- S represents possible states
- A represents available actions
- O represents environment observations

At time step t , the agent produces an action

$$a_t = \text{LLM}(c_t)$$

where c_t represents the context including task instructions, previous actions, and observations.

The objective is to maximize the probability of successfully completing tasks of increasing horizon length.

IV. CHALLENGES IN LONG-HORIZON TASKS

A. Error Accumulation

Small reasoning mistakes propagate across sequential decisions, leading to failure states that are difficult to recover from.

B. Context Window Limitations

LLMs have finite context windows. As tasks grow longer, earlier reasoning steps may be truncated.

C. Repeated Failure Patterns

Agents often repeat the same actions even when those actions repeatedly fail.

Example:

```
Step 1: check_permissions(file.txt)
Step 2: chmod 644 file.txt
Step 3: verify_permissions(file.txt)
Step 4: chmod 644 file.txt
Step 5: chmod 644 file.txt
```

The agent fails to recognize the underlying issue.

V. PROPOSED METHOD

Our framework introduces five components.

A. Trace Collection

During task execution, the agent records reasoning, actions, and observations.

$$Trace = \{(r_t, a_t, o_t)\}_{t=1}^T$$

B. Failure Analysis

If a task fails, a failure analyzer examines the trace to identify patterns such as repeated actions or incorrect reasoning.

C. Strategy Generation

A secondary LLM generates corrective strategies describing what went wrong and how to fix it.

D. Strategy Memory

Strategies are stored in a persistent database along with associated failure patterns.

E. Strategy-Guided Execution

When a similar failure pattern is detected in future tasks, the agent retrieves relevant strategies and integrates them into its reasoning prompt.

Algorithm 1 Self-Improving Agent

```
1: for each task  $T$  do
2:   Execute task using LLM agent
3:   Record execution trace
4:   if task fails then
5:     Analyze trace
6:     Generate strategy using LLM
7:     Store strategy in memory
8:   end if
9:   Retrieve relevant strategies for next task
10: end for
```

VI. ALGORITHM

VII. EXPERIMENTAL SETUP

A. Datasets

We evaluate using two benchmarks.

AgentBench provides environments including OS interaction, database queries, and programming tasks.

WebArena simulates realistic web navigation tasks requiring sequential actions.

B. Metrics

We measure:

- Task success rate
- Success vs horizon length
- Failure mode distribution

C. Baselines

We compare against:

- ReAct agents
- Plan-and-Act agents
- Standard LLM agents

VIII. COMPUTE REQUIREMENTS

The system operates primarily at inference time.

Models used include:

- LLaMA-3-8B
- Mistral-7B
- GPT-4 API

Experiments require approximately 100–200 tasks with 5–10 steps per task.

IX. EXPECTED RESULTS

We expect our method to:

- Improve task success rates
- Reduce repeated failure patterns
- Improve reasoning efficiency in long tasks

X. LIMITATIONS

The approach depends on the quality of failure analysis and strategy generation. Incorrect strategies may propagate through the memory system.

Future work could incorporate reinforcement learning or meta-learning techniques.

XI. CONCLUSION

Long-horizon reasoning remains a major challenge for LLM agents. We introduced a self-improving agent framework that learns from execution traces without retraining the underlying model.

By combining failure analysis, strategy generation, and persistent memory, the agent can progressively improve performance across tasks. This approach provides a promising direction for building reliable autonomous AI systems.

REFERENCES

- [1] Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR 2023.
- [2] Wang et al. “Plan-and-Solve Prompting.” ACL 2023.
- [3] Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” NeurIPS 2023.
- [4] Wang et al. “Voyager: An Open-Ended Embodied Agent with Large Language Models.” ICML 2023.
- [5] Liu et al. “AgentBench: Evaluating LLMs as Agents.” NeurIPS 2023.
- [6] Zhou et al. “WebArena: A Realistic Web Environment for Building Autonomous Agents.” ICLR 2024.